

AI MT-1 Key

=== SECTION A ANSWERS ===

- 1) $S \rightarrow G$
- 2) $S \rightarrow G$
- 3) Yes
- 4) No
- 5) $S \rightarrow A \rightarrow C \rightarrow G : 7$

=== SECTION B ANSWERS ===

```
1)
1. from copy import deepcopy
2.
3. class State:
4.     def __init__(self, grid):
5.         self.grid = grid
6.         self.n = len(grid)
7.
8.     def goalTest(self):
9.         for r in range(self.n):
10.            for c in range(self.n):
11.                if self.grid[r][c] != 0:
12.                    return False
13.        return True
14.
15.    def moveGen(self):
16.        children = []
17.        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right
18.
19.        for r in range(self.n):
20.            for c in range(self.n):
21.
22.                new_grid = deepcopy(self.grid)
23.                for dr, dc in directions:
24.                    new_r = r + dr
25.                    new_c = c + dc
26.                    if 0 <= new_r < self.n and 0 <= new_c < self.n:
27.                        new_grid[new_r][new_c] = 1 - new_grid[new_r][new_c]
28.
29.                children.append(((r, c), State(new_grid)))
30.        return children
```

2)

#of states = $7! / (3! * 3! * 1!) = 140$ (Tight upper bound)
Upper bound = $7!$

A heuristic should estimate how far we are from the goal.

Current State = [B][B][B][W][W][W][E]

Goal State = [W][W][W][B][B][B][E]

Simple heuristic: $h(n)$ = number of mismatched tiles, ignoring empty cell

Example: 3 Blacks in wrong half + 3 Whites in wrong half = 6.

Another Heuristic: $h(n)$ = for each Black tile, count how many White tiles are to its right and sum.

Example: Each of the 3 Blacks has 3 Whites to its right $\rightarrow 3 \times 3 = 9$

3)

```
1. count  $\leftarrow$  -1
2. path  $\leftarrow$  empty list
3. depthBound  $\leftarrow$  0
4. repeat
5.     previousCount  $\leftarrow$  count
6.     (count, path)  $\leftarrow$  DB-DFS(S, depthBound)
7.     depthBound  $\leftarrow$  depthBound + 1
8. until (path is not empty) or (previousCount = count)
9. return path
```

DB-DFS(S, depthBound)

```
1. count  $\leftarrow$  0
2. OPEN  $\leftarrow$  [(S, null, 0)]           // List of (Node, Parent, Depth)
3. CLOSED  $\leftarrow$  empty list
4. while OPEN is not empty do
5.     nodePair  $\leftarrow$  head(OPEN)
6.     (N, _, depth)  $\leftarrow$  nodePair
7.     if GoalTest(N) = true then
8.         return (count, ReconstructPath(nodePair, CLOSED))
9.     else
10.        CLOSED  $\leftarrow$  nodePair : CLOSED
11.        if depth < depthBound then
12.            children  $\leftarrow$  MoveGen(N)
13.            newNodes  $\leftarrow$  RemoveSeen(children, OPEN, CLOSED)
14.            newPairs  $\leftarrow$  MakePairs(newNodes, N, depth + 1)
15.            OPEN  $\leftarrow$  newPairs ++ tail(OPEN)
16.            count  $\leftarrow$  count + length(newPairs)
17.        else
18.            OPEN  $\leftarrow$  tail(OPEN)
19. return (count, empty list)
```

//Or description

Complexity:

$$T(d) = (d+1) \cdot b^0 + d \cdot b^1 + (d-1) \cdot b^2 + \dots + 1 \cdot b^d$$

$$T(d) = O(b^d)$$



Subject: Artificial Intelligence

Date: __/08/2025

Exam: AY_25-26_E3SEM1_Monthly Test-I

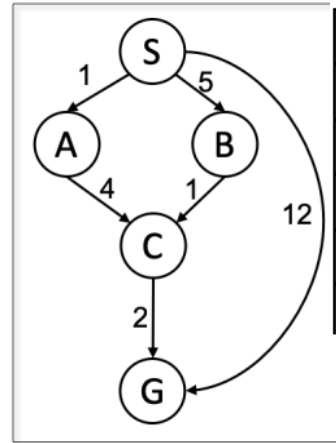
Subject Code: CS3103

Time: 45 Mins.

Max Marks: 15

Section - A (5 Marks)

Consider the search graph shown on the right. S is the start state and G is the goal state. Edges are annotated with their cost. The table shows the values for each node for two different heuristic functions: h1 and h2. Answer the following: (If there are ties, assume alphabetical tie breaking i.e., nodes for states earlier in the alphabet are expanded first).



node	h1	h2
S	6	20
A	5	8
B	2	6
C	1	10
G	0	0

1. The path that will be returned by Depth-first graph search, implemented with OPEN and CLOSED lists, ignoring edge costs is _____.
2. The path that will be returned by Best-first search using heuristic h1 is _____.
3. Is heuristic h1 admissible? _____.
4. Is heuristic h2 admissible? _____.
5. The path that will be returned by A* search using an admissible heuristic is _____.

Section - B (10 Marks - Answer any two)

1. The *AllOut* game is played on a 5 by 5 board. Each square can be in two positions, *ON* or *OFF*. The initial state is some state, where at least one square is *ON*. The moves constitute of clicking on a particular square. The effect of the click is to toggle the positions of its four neighbouring squares. The task is to bring all squares to *OFF* position. Pose the above problem as a state space search problem and suggest a reasonable representation in any programming language. [5]
2. Consider a sliding tile puzzle with six tiles (three black and three white) in a linear tray which can hold seven tiles. The following depicts the initial configuration:

[B][B][B][W][W][W][E]

where [B] represents a black tile, [W] a white one and [E] the 'empty' cell. The puzzle has the following two moves: (1) A tile may move into an adjacent empty cell with unit cost; and (2) A tile may hop over one or two tiles into the empty cell with cost

equal to the number of tiles hopped over. Thus, the initial configuration has the following three immediate successors:

[B][B][B][W][W][E][W] (cost = 1)

[B][B][B][W][E][W][W] (cost = 1)

[B][B][B][E][W][W][W] (cost = 2)

The goal is to have all of the white tiles to the left of all of the black ones. It is unimportant where the empty cell is. Your task is to set this problem up for solution by algorithm A.

Give an tight upper bound on the size of the search space in terms of the number of states by ignoring the fact that identity of the three tiles of each color is unimportant. [2]

Describe a heuristic function (h) appropriate for this problem. [3]

3. Explain the Iterative Deepening DFS algorithm and analyze its time complexity. [3+2]
